

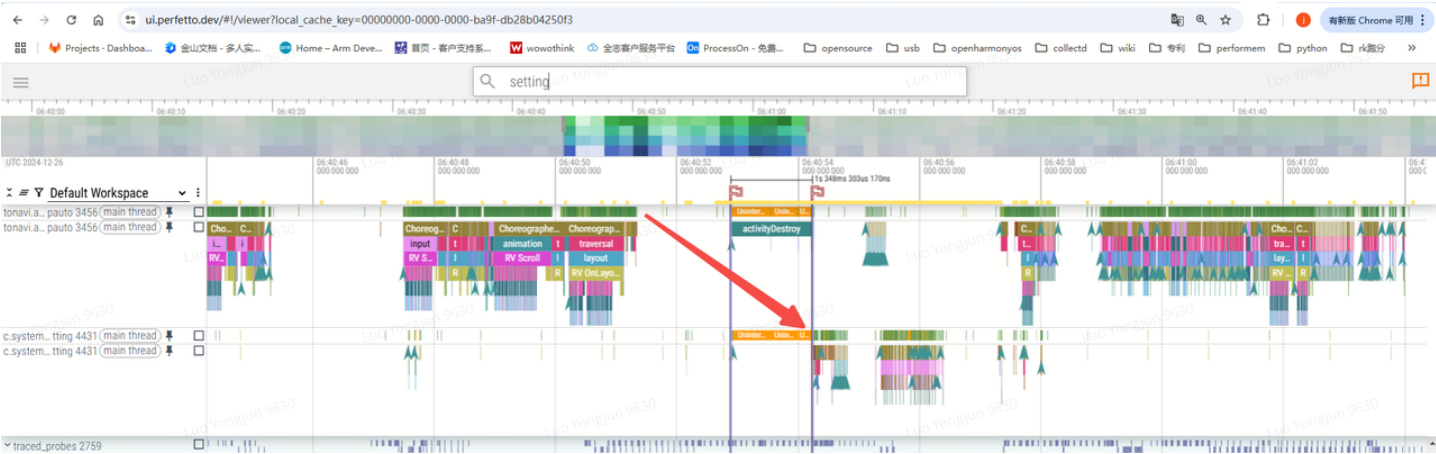
航盛CCS2.0 IO导致卡顿分析

项目信息

- 客户名称：扬州航盛
- 客户项目：CCS2.0+ & P32S
- 硬件环境：
- 软件版本：
- FAE：朱杰
- 硬件：詹茂晴
- support问题票：<https://support.semidrive.com/issue/bugs/view/15510>
- <https://support.semidrive.com/issue/bugs/view/15522>

问题描述

问题前期定位，已经通过perfetto工具定位分析到是IO导致的卡顿了，如下图所示：



从perfetto工具可以看出，从地图APP切换到setting APP时，存在长达1.3s的IO等待，这个就是UI掉帧，在用户角度表现为卡顿的原因。基于此，测试iozone性能，有以下发现：

客户data分区22GB，刚刷机时，data分区空闲15G左右，通过以下步骤可以在客户机器上稳定复现问题：

- 上电开机启动，联网，下载离线地图，把data分区100%占用
- 通过地图UI，删除下载的离线地图文件，使data分区空闲12G左右，可以空闲更多
- 使用以下IOZONE指令测试IO性能

```

1 //把cpu频率设置到最高，Android系统stop掉，写data分区的服务stop
2 echo 1600000 > /sys/devices/system/cpu/cpu0/cpufreq/scaling_min_freq
3 cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_min_freq
4 am broadcast -a android.intent.action.test --ei tiaoshi 25
5 stop persistlog-xlog
6 stop sdlogd
7 stop
8
9 iotestresult_apl.xls resource-cache warning.rgb
-i 0 -s 8G -r 2048k -I -f /data/test -Rb iotestresult_apl.xls

```

4. iotest性能结果对比

第一次iotest结果：

```

iotestresult_apl.xls resource-cache warning.rgb
-i 0 -s 8G -r 2048k -I -f /data/test -Rb iotestresult_apl.xls
Iotest: Performance Test of File I/O
Version $Revision: 3.489 $
Compiled for 64 bit mode.
Build: linux-arm

Contributors:William Norcott, Don Capps, Isom Crawford, Kirby Collins
Al Slater, Scott Rhine, Mike Wisner, Ken Goss
Steve Landherr, Brad Smith, Mark Kelly, Dr. Alain CYR,
Randy Dunlap, Mark Montague, Dan Million, Gavin Brebner,
Jean-Marc Zucconi, Jeff Blomberg, Benny Halevy, Dave Boone,
Erik Habbita, Kris Strecker, Walter Wong, Joshua Root,
Fabrice Bacchella, Zhenghua Xue, Qin Li, Darren Sawyer,
Vangel Bojaxhi, Ben England, Vikentsi Lapa,
Alexey Skidanov, Sudhir Kumar.

Run began: Fri Feb 14 11:01:20 2025

File size set to 8388608 kB
Record Size 2048 kB
O_DIRECT feature enabled
Excel chart generation enabled
Command line used: ./iotest -i 0 -s 8G -r 2048k -I -f /data/test -Rb iotestresult_apl.xls
Output is in kBytes/sec
Time Resolution = 0.000001 seconds.
Processor cache size set to 1024 kBytes.
Processor cache line size set to 32 bytes.
File stride size set to 17 * record size.

      kB  reclen  write  rewrite  read  reread  random  random  bkwd  record  stride  fwrite  frewrite  fread  freread
8388608  2048    54283   99197         99197         99197         99197         99197         99197         99197         99197         99197         99197         99197

iotest test complete.
Excel output is below:

"writer report"
"2048"
"8388608" 54283

```

第二次iotest结果：

```

Vangel Bojaxhi, Ben England, Vikentsi Lapa,
Alexey Skidanov, Sudhir Kumar.

Run began: Fri Feb 14 12:27:38 2025

File size set to 8388608 kB
Record Size 2048 kB
O_DIRECT feature enabled
Excel chart generation enabled
Command line used: iotest -i 0 -s 8G -r 2048k -I -f /data/test -Rb iotestresult_apl.xls
Output is in kBytes/sec
Time Resolution = 0.000001 seconds.
Processor cache size set to 1024 kBytes.
Processor cache line size set to 32 bytes.
File stride size set to 17 * record size.

      kB  reclen  write  rewrite  read  reread  random  random  bkwd  record  stride  fwrite  frewrite  fread  freread
8388608  2048   125342  159169         159169         159169         159169         159169         159169         159169         159169         159169         159169         159169

iotest test complete.
Excel output is below:

"writer report"
"2048"
"8388608" 125342

"Re-writer report"
"2048"
"8388608" 159169

```

可以看出，两次iotest性能存在巨大差异，下面对iotest性能差异进行分析

问题分析

工具使用说明

在iozone测试过程中，通过blktrace和trace-cmd工具抓取IO的行为进行分析，抓取指令如下：

```
1 blktrace -d /dev/block/mmcblk0p45 -a WRITE -o /hsalog/blktrace -w 350
2 trace-cmd record -e block:block_rq_complete -e block:block_rq_issue -e
  mmc:mmc_request_done -e mmc:mmc_request_start -e irq:irq_handler_entry -f
  'irq == 21' -e irq_handler_exit -f 'irq == 21' -o /hsalog/blktrace/trace.dat
```

iozone的blktrace数据对比

因为两次iozone存在性能差异，因此，我们先使用blktrace对iozone的测试过程中的io生命周期进行统计分析，抓取到的数据如下：

 fast.zip

 slow.zip

通过blkparse和btt命令，可以对blktrace抓到的数据进行汇总和详细分析

```
1 blkparse -i blktrace.blktrace.* -d blktrace.data.bin -o blktrace.data.txt
2 btt -i blktrace.data.bin > sum.txt
```

我们对上述抓到的文件使用工具进行分析后，得出比较容易看到的数据对比如下：

1. btt的总结报告sum.txt

All Devices					
	ALL	MIN	AVG	MAX	N
Q2Q	0.000000001	0.004515632	2.008709000	87987	
Q2G	0.000002333	0.004008156	2.126951334	72964	
S2G	0.001462333	0.123965388	2.126940667	2352	
G2I	0.000000666	0.000016341	0.008117333	72964	
Q2M	0.000000333	0.000000871	0.000005667	15024	
I2D	0.000004667	0.162174877	4.029561667	7460	
M2D	0.000013000	0.000805818	0.081425666	15024	
D2C	0.000059333	0.001088200	0.657173334	5621	
Q2C	0.000082333	0.427850973	5.300378001	21997	
Device Overhead					

All Devices					
	ALL	MIN	AVG	MAX	N
Q2Q	0.000000001	0.004658235	5.113417667	80663	
Q2G	0.000002333	0.007049764	5.112472667	70380	
S2G	0.007439333	0.225518260	5.112461667	2196	
G2I	0.000000667	0.000018114	0.007144334	70380	
Q2M	0.000000333	0.000000889	0.000019000	10284	
I2D	0.000004666	0.034930176	5.339536334	4844	
M2D	0.000013333	0.003641362	0.081280667	10284	
D2C	0.000059667	0.002666377	1.200203000	3782	
Q2C	0.000084333	0.826401828	6.775697334	20166	
Device Overhead					

从btt工具的报告可以看出，D2C存在性能差异，而这个一般表示存储设备硬件可能存在性能瓶颈，各个阶段IO含义说明如下：

```
1  Q----->G----->I----->M----->D-----
   ----->C
2  |-Q time-|-Insert time-|
3  |----- merge time -----|-merge with other IO|
4  |-----scheduler time time-----|---
   driver,adapter,storagetime--|
5  |----- await time in iostat output -----
   -----|
```

- 6 Q2Q – time between requests sent to the block layer
- 7 Q2G – time from a block I/O is queued to the time it gets a request allocated for it
- 8 G2I – time from a request is allocated to the time it is Inserted into the device's queue
- 9 Q2M – time from a block I/O is queued to the time it gets merged with an existing request
- 10 I2D – time from a request is inserted into the device's queue to the time it is actually
- 11 issued to the device
- 12 M2D – time from a block I/O is merged with an exiting request until the request is issued
- 13 to the device
- 14 D2C – service time of the request by the device
- 15 Q2C – total time spent in the block layer for a request

blk事件对应的跟踪函数如下表所示：

blktrace事件	静态跟踪点	动态跟踪点
Q	block_bio_queue	xxx
G	block_getrq	xxx
I	block_rq_insert	xxx
M	block_bio_backmerge	xxx
D	block_rq_issue	blk_peek_request
C	block_rq_complete	blk_complete_request blk_update_request

2. io的完整生命周期blktrace.data.txt

从btt的总结信息可以看出，异常时，D2C的平均耗时是2.66ms，最大耗时为1200ms，而正常的D2C平均耗时为1.08ms，最大耗时为657ms，通过blktrace.data.txt找出最大耗时情况如下：

iozone_slow.txt														
6427	259,13	1	390	1.049022000	1669	C	W	26261760	+	2048	[0]			
6428	259,13	1	390	1.049022000	1669	C	W	26261760	+	2048	[0]			
6429	259,13	1	391	1.049308334	1669	D	W	26264832	+	1024	[mmcqd/0]			
6430	259,13	1	391	1.049308334	1669	D	W	26264832	+	1024	[mmcqd/0]			
6431	259,13	1	391	1.049308334	1669	D	W	26264832	+	1024	[mmcqd/0]			
6432	259,13	1	391	1.049308334	1669	D	W	26264832	+	1024	[mmcqd/0]			
6433	259,13	1	392	1.052032334	1669	C	W	26263808	+	1024	[0]			
6434	259,13	1	392	1.052032334	1669	C	W	26263808	+	1024	[0]			
6435	259,13	1	392	1.052032334	1669	C	W	26263808	+	1024	[0]			
6436	259,13	1	392	1.052032334	1669	C	W	26263808	+	1024	[0]			
6437	259,13	1	393	1.052043000	1669	D	W	26265856	+	1024	[mmcqd/0]			
6438	259,13	1	393	1.052043000	1669	D	W	26265856	+	1024	[mmcqd/0]			
6439	259,13	1	393	1.052043000	1669	D	W	26265856	+	1024	[mmcqd/0]			
6440	259,13	1	393	1.052043000	1669	D	W	26265856	+	1024	[mmcqd/0]			
6441	259,13	1	394	1.275503334	1669	C	W	26263808	+	2048	[0]			
6442	259,13	1	394	1.275503334	1669	C	W	26263808	+	2048	[0]			
6443	259,13	1	394	1.275503334	1669	C	W	26263808	+	2048	[0]			
6444	259,13	1	394	1.275503334	1669	C	W	26263808	+	2048	[0]			
6445	259,13	1	395	1.275811667	1669	D	W	26266880	+	1024	[mmcqd/0]			
6446	259,13	1	395	1.275811667	1669	D	W	26266880	+	1024	[mmcqd/0]			
6447	259,13	1	395	1.275811667	1669	D	W	26266880	+	1024	[mmcqd/0]			
6448	259,13	1	395	1.275811667	1669	D	W	26266880	+	1024	[mmcqd/0]			
6449	179,0	0	1159	2.007623334	1	A	WS	18616928	+	8	<- (259,13) 4108640			
6450	179,0	0	1159	2.007623334	1	A	WS	18616928	+	8	<- (259,13) 4108640			
6451	179,0	0	1159	2.007623334	1	A	WS	18616928	+	8	<- (259,13) 4108640			
6452	179,0	0	1159	2.007623334	1	A	WS	18616928	+	8	<- (259,13) 4108640			
6453	259,13	0	1160	2.007626334	1	Q	WS	18616928	+	8	[init]			
6454	259,13	0	1160	2.007626334	1	Q	WS	18616928	+	8	[init]			
6455	259,13	0	1160	2.007626334	1	Q	WS	18616928	+	8	[init]			
6456	259,13	0	1160	2.007626334	1	Q	WS	18616928	+	8	[init]			
6457	259,13	0	1161	2.007643667	1	G	WS	18616928	+	8	[init]			
6458	259,13	0	1161	2.007643667	1	G	WS	18616928	+	8	[init]			
6459	259,13	0	1161	2.007643667	1	G	WS	18616928	+	8	[init]			
6460	259,13	0	1161	2.007643667	1	G	WS	18616928	+	8	[init]			
6461	259,13	0	1162	2.007653667	1	I	WS	18616928	+	8	[init]			
6462	259,13	0	1162	2.007653667	1	I	WS	18616928	+	8	[init]			
6463	259,13	0	1162	2.007653667	1	I	WS	18616928	+	8	[init]			
6464	259,13	0	1162	2.007653667	1	T	WS	18616928	+	8	[init]			
6465	259,13	1	396	2.216119001	1669	C	W	26265856	+	1024	[0]			
6466	259,13	1	396	2.216119001	1669	C	W	26265856	+	1024	[0]			
6467	259,13	1	396	2.216119001	1669	C	W	26265856	+	1024	[0]			
6468	259,13	1	396	2.216119001	1669	C	W	26265856	+	1024	[0]			
6469	259,13	1	397	2.216154667	1669	D	WS	18616928	+	8	[mmcqd/0]			
6470	259,13	1	397	2.216154667	1669	D	WS	18616928	+	8	[mmcqd/0]			
6471	259,13	1	397	2.216154667	1669	D	WS	18616928	+	8	[mmcqd/0]			
6472	259,13	1	397	2.216154667	1669	D	WS	18616928	+	8	[mmcqd/0]			
6473	259,13	1	398	3.415972334	1669	C	W	26265856	+	2048	[0]			
6474	259,13	1	398	3.415972334	1669	C	W	26265856	+	2048	[0]			
6475	259,13	1	398	3.415972334	1669	C	W	26265856	+	2048	[0]			
6476	259,13	1	398	3.415972334	1669	C	W	26265856	+	2048	[0]			
6477	259,13	1	399	3.416275001	1669	D	W	26267904	+	1024	[mmcqd/0]			
6478	259,13	1	399	3.416275001	1669	D	W	26267904	+	1024	[mmcqd/0]			
6479	259,13	1	399	3.416275001	1669	D	W	26267904	+	1024	[mmcqd/0]			

而正常情况下，同样写1024 block，耗时最大是600~700ms左右

[iozone_fast.txt](#)

[iozone_slow.txt](#)

[sum_fast.txt](#)

[sum_slow.txt](#)

blktrace结合trace-cmd工具定位D2C流程慢

同理，先通过blktrace抓到日志后，处理日志信息，获取到异常IO情况，日志数据如下：

[blktrace.zip](#)

对日志进行处理后，找到异常IO如下：

结合系统负载情况和每次测试情况，可以得出结论，应该属于第二种情况，如果需要证实，则需要使用trace-cmd工具中，加上系统开关中断的事件一起监控，但从目前分析，应该不需要这个操作。基本可以以下结论是emmc慢导致的IO慢，和文件系统、软件配置等无关。

问题结论

emmc器件问题慢导致，需要emmc原厂跟进澄清